METASYNC

A DISTRIBUTED METADATA & FILE REPLICATION MANAGEMENT SYSTEM

DATABASE MANAGEMENT SYSTEMS LAB PROJECT

Team Member	Registration No.
Sara Jain	24BCE2360
Vispute Shreya Ganesh	24BCE0563

Executive Summary

MetaSync is a distributed file-management prototype that separates the physical storage of file payloads from the database-managed metadata required to find, version, replicate, audit, and retrieve those files. The backend is implemented with Node.js/Express and communicates with a local MongoDB database at 127.0.0.1:27017/metasync. Actual file bytes are stored in simulated storage-node directories, while MongoDB maintains structured documents describing users, files, versions, replicas, operations, permissions, sessions, storage nodes, and audit events.

The project demonstrates an end-to-end workflow: server initialization, database connectivity, account provisioning, authentication, storage-node organization, file upload, checksum generation, metadata registration, version control, cross-node replication, physical payload management, retrieval, and audit logging. It therefore combines practical Operating Systems ideas such as file-system abstraction, storage virtualization, integrity checking, and redundancy with DBMS concepts such as document persistence, cross-document references, metadata management, and write traceability.

Scope of this report. The report documents the supplied implementation evidence. Screenshots are placed directly after the relevant functionality so that each described feature is immediately supported by project evidence.

1. Applied Concepts — DBMS

MetaSync is intentionally designed around database management concepts. The following DBMS concepts map directly to the implementation choices demonstrated by the project.

Domain	Concept	Application in MetaSync
DBMS	Decoupled Data Architecture	File bytes are kept in simulated node directories, while MongoDB stores logical metadata, versions, replica mappings, and audit information.
DBMS	Schema Relations & Referential Integrity	ObjectId references connect users, files, versions, replicas, and audit records, creating explicit relationships between independently stored documents.
DBMS	Audit Logging & Write Traceability	The auditlogs collection records actor, action, file/version references, details, and UTC timestamps for important operations.

1.1 DBMS Perspective

From the DBMS perspective, MongoDB acts as the system-of-record for state that must survive beyond a single request. The project separates concerns into collections instead of mixing user data, file metadata, historical versions, and operational events. The use of ObjectId references provides relationship semantics similar to foreign-key references in a relational design, even though MongoDB itself is document-oriented.

The metadata design also demonstrates why databases are useful for distributed storage systems: a database can answer questions such as who owns a file, which version is active, which nodes hold replicas, what checksum is expected, and what operations have occurred without scanning every physical storage directory.

1.2 DBMS Data Flow

Step	Operation	Primary state involved
1	User authenticates	users / sessions
2	File selected and target node chosen	dashboard / storage node
3	Payload written to physical node	storage/nodeX
4	Checksum and version created	fileversions
5	Logical file metadata updated	files
6	Replica locations registered	replicas
7	Operation recorded	auditlogs
8	File retrieved through browser	logical metadata → object path → response

Functionalities

1 Application Startup and MongoDB Connection

MetaSync starts an Express server on port 3000 and connects to the local MongoDB service before handling database based requests.

The connection string selects the metasync database, where all file metadata and user related information is stored.

MongoDB now runs as a Windows service, so the application can be restarted using npm run dev without manually starting mongod.

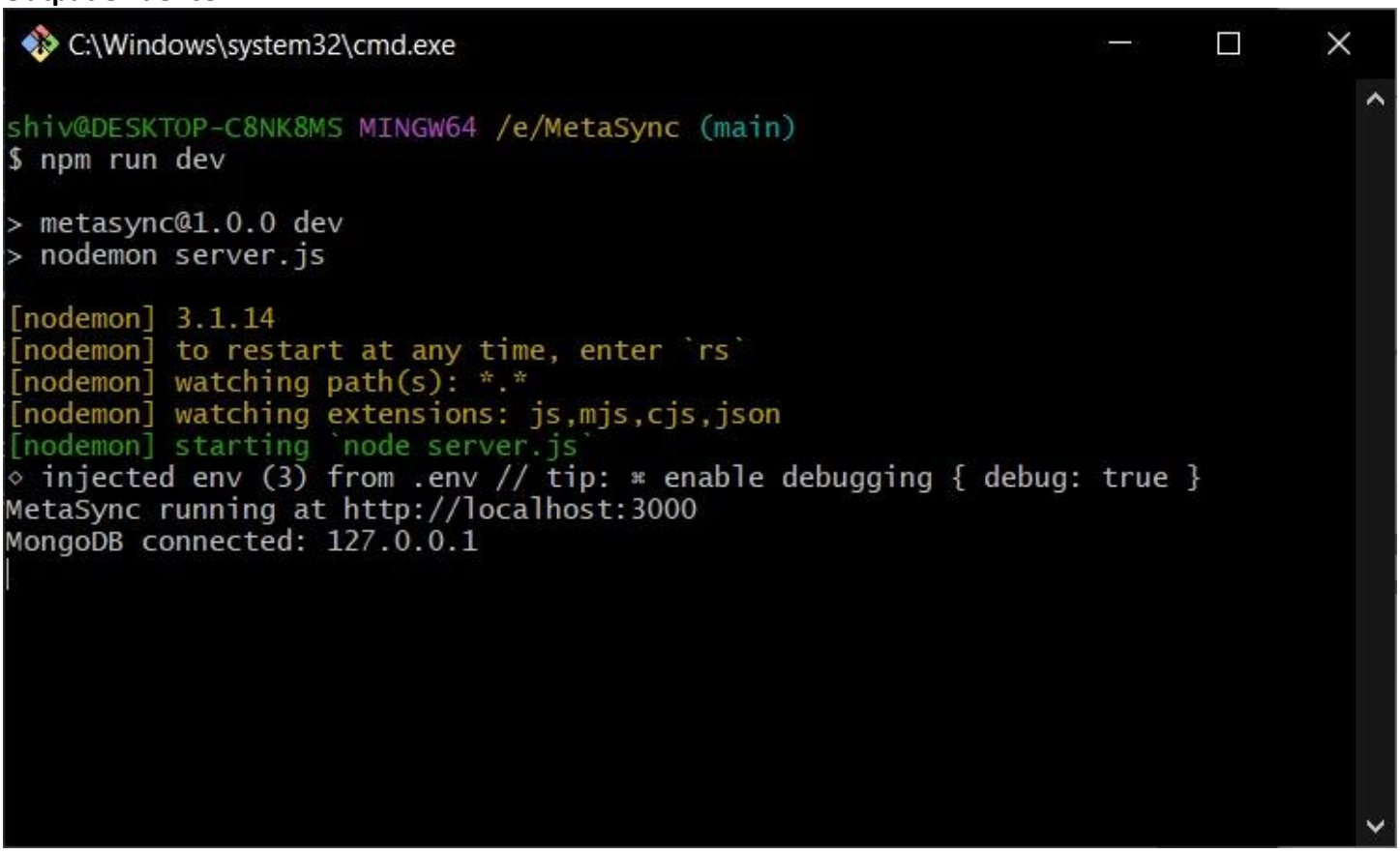
This proves that the application layer and database layer are connected.

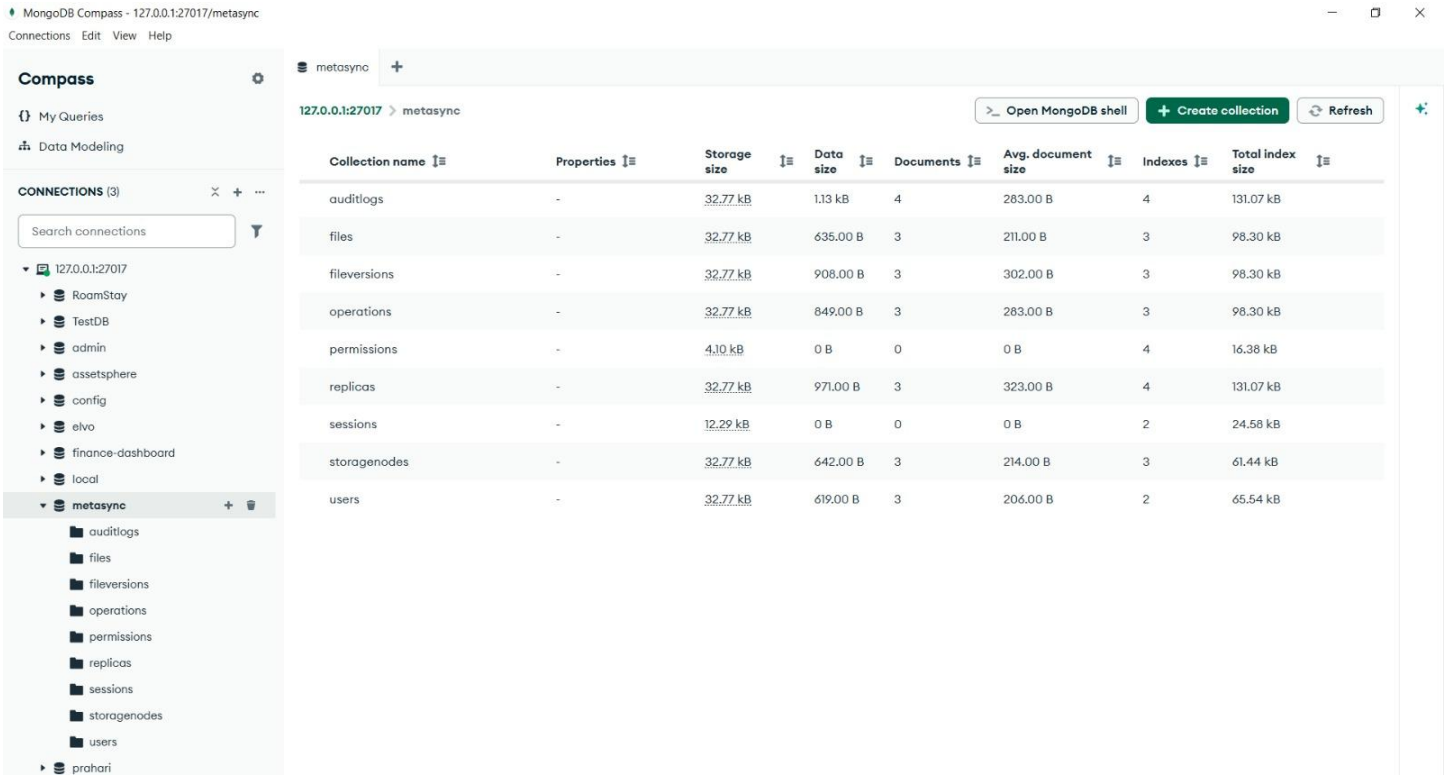
Main code snippet

```
await mongoose.connect(process.env.MONGO_URI);
console.log("MongoDB connected");

app.listen(PORT, () => {
  console.log(`MetaSync running at http://localhost:${PORT}`);
});
```

Output evidence





Collection name	Properties	Storage size	Data size	Documents	Avg. document size	Indexes	Total index size
auditlogs	-	32.77 kB	1.13 kB	4	283.00 B	4	131.07 kB
files	-	32.77 kB	635.00 B	3	211.00 B	3	98.30 kB
fileversions	-	32.77 kB	908.00 B	3	302.00 B	3	98.30 kB
operations	-	32.77 kB	849.00 B	3	283.00 B	3	98.30 kB
permissions	-	4.10 kB	0 B	0	0 B	4	16.38 kB
replicas	-	32.77 kB	971.00 B	3	323.00 B	4	131.07 kB
sessions	-	12.29 kB	0 B	0	0 B	2	24.58 kB
storagenodes	-	32.77 kB	642.00 B	3	214.00 B	3	61.44 kB
users	-	32.77 kB	619.00 B	3	206.00 B	2	65.54 kB

2 User Registration

A new user can create a MetaSync account using the registration form.

The password is encrypted with bcrypt before it is stored, so plaintext passwords are never saved in MongoDB.

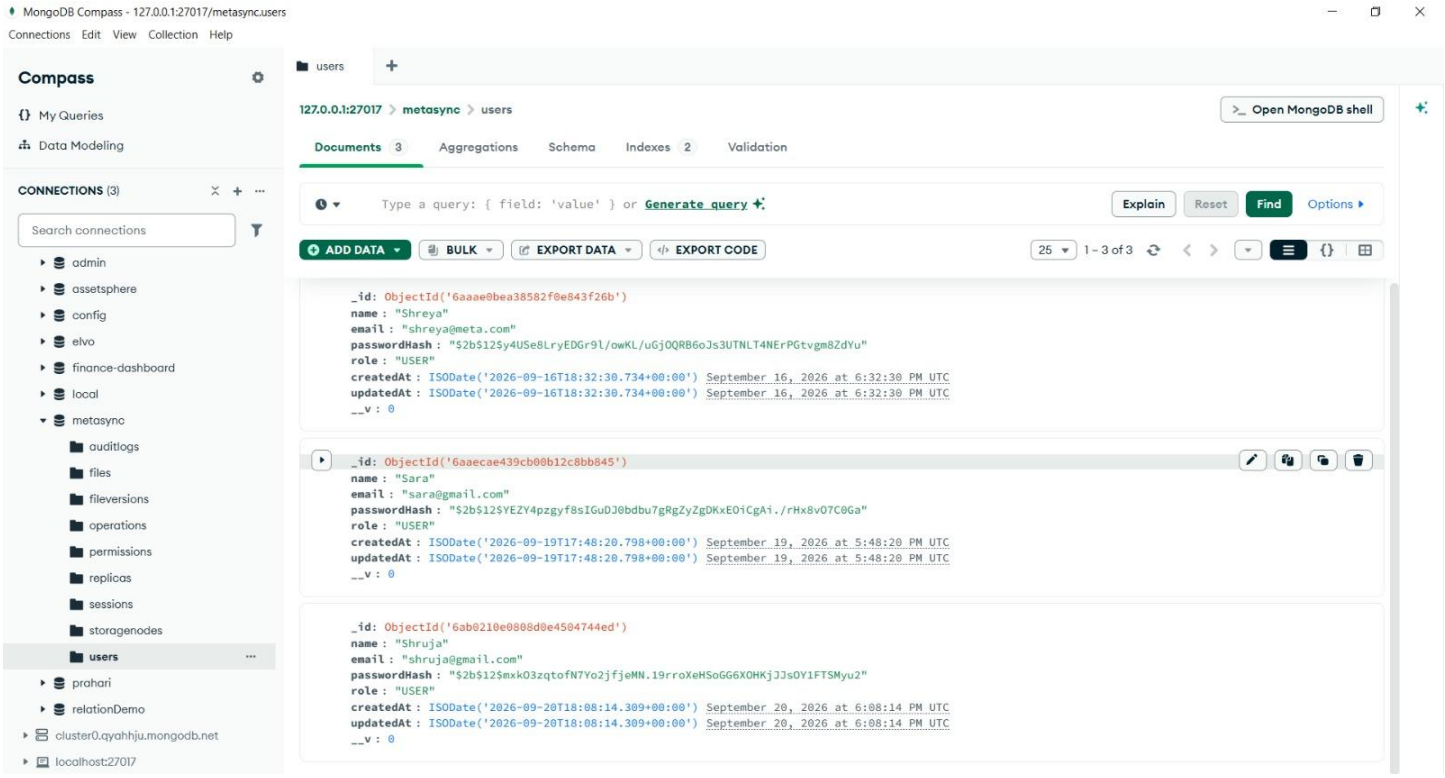
The user record is stored in the users collection and becomes the owner of files uploaded from that account.

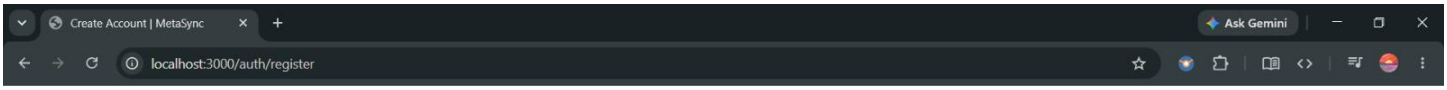
This feature establishes identity and ownership for all later operations.

Main code snippet

```
const hashedPassword = await bcrypt.hash(password, 12);
await User.create({
  name,
  email,
  password: hashedPassword
});
res.redirect("/auth/login");
```

Output evidence





Create MetaSync Account

Full name

Email address

Password

Create account

Already registered? Log in

3 Login Session and Logout

Registered users can log in with their email and password.

bcrypt compares the entered password with the hashed value stored in the users collection.

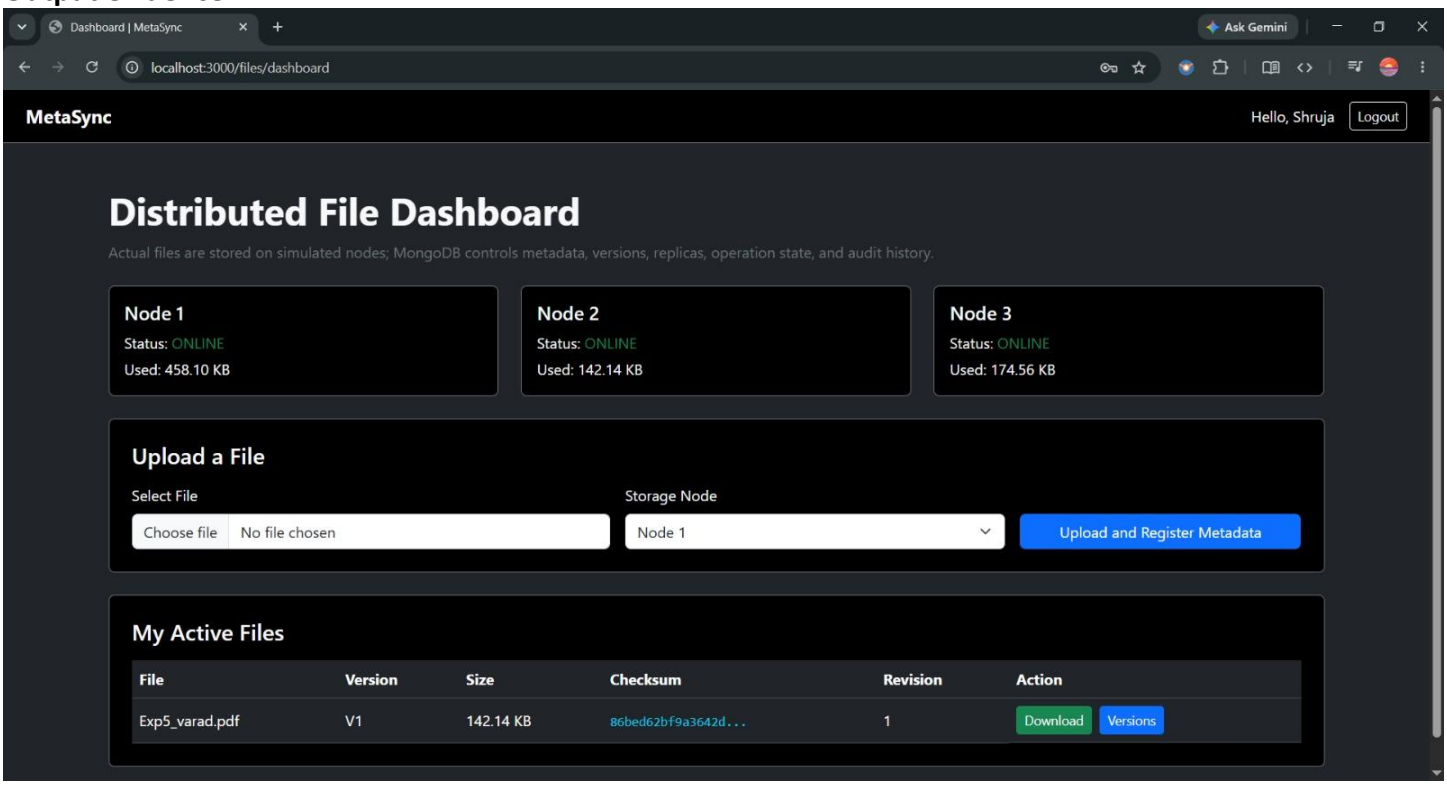
After successful authentication, Express session stores the user id and protects dashboard routes.

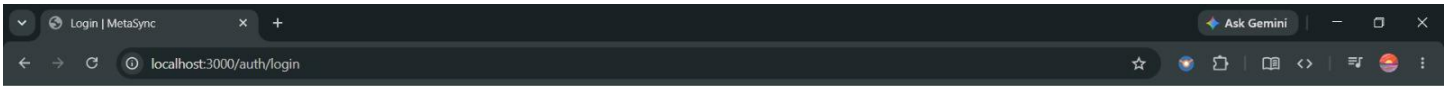
Logout destroys the session so protected data cannot be accessed from that browser session.

Main code snippet

```
const user = await User.findOne({ email });
const valid = await bcrypt.compare(password, user.password);
if (!valid) return res.redirect("/auth/login");
req.session.userId = user._id;
res.redirect("/files/dashboard");
```

Output evidence





Login to MetaSync

Email address

Password

Login

New user? Create an account

4 Storage Node Dashboard

The dashboard visualizes three simulated storage nodes: Node 1, Node 2, and Node 3.

Each node exposes its online status and used storage value.

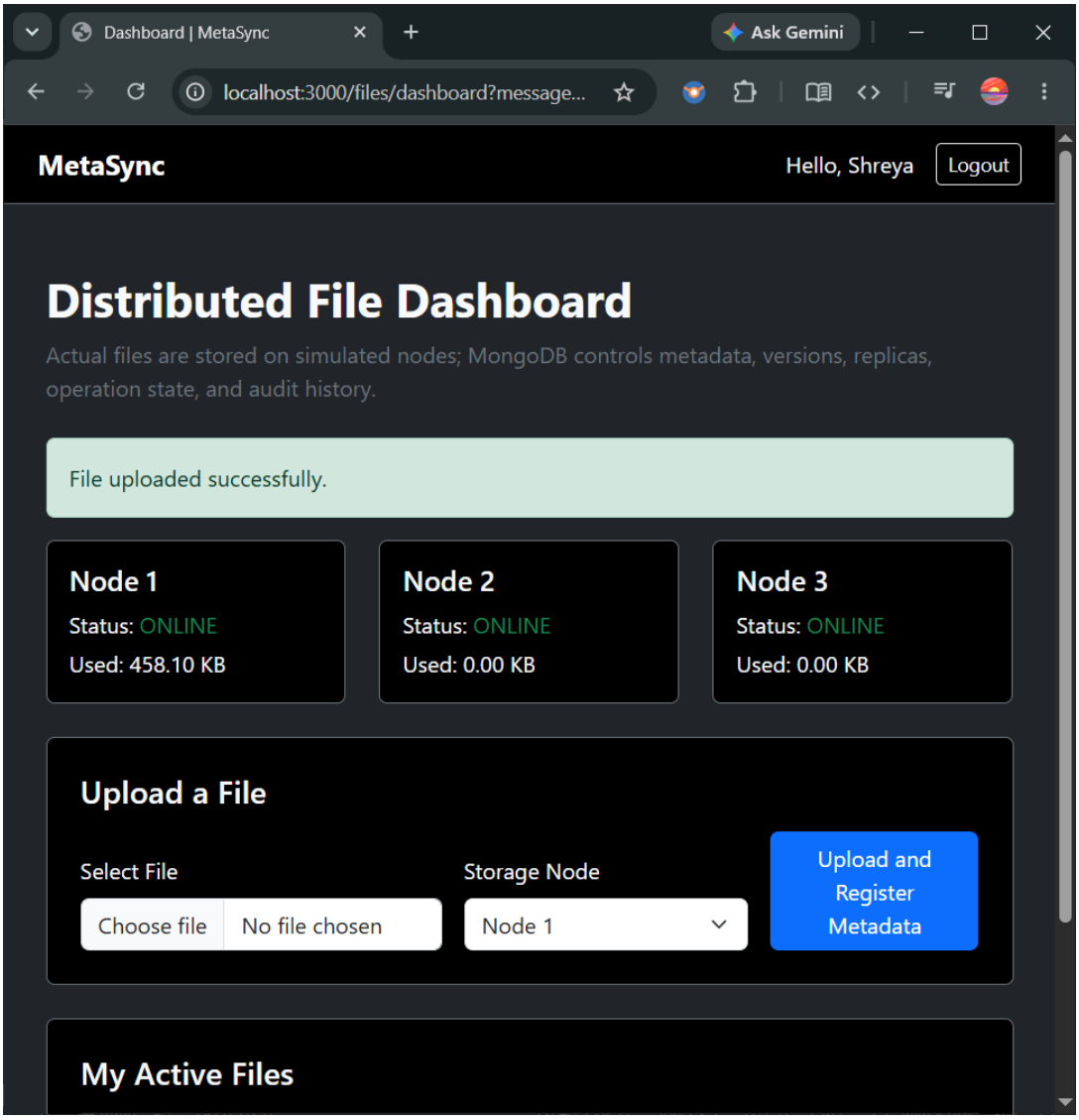
The nodes represent the physical distribution layer, while MongoDB stores the control metadata.

This gives a clear distributed storage view rather than storing every file in one application folder.

Main code snippet

```
const nodes = await StorageNode.find().sort({ nodeNumber: 1 });
res.render("files/dashboard", {
  nodes,
  user: req.user
});
```

Output evidence



Dashboard output showing Node 1, Node 2, Node 3, and a successful upload.

5 File Upload and Metadata Registration

The user selects a file and a destination node from the dashboard.

Multer receives the file and the controller saves the physical bytes in the chosen node folder.

At the same time, MetaSync creates a File document containing ownership, original name, checksum, storage location, and current version.

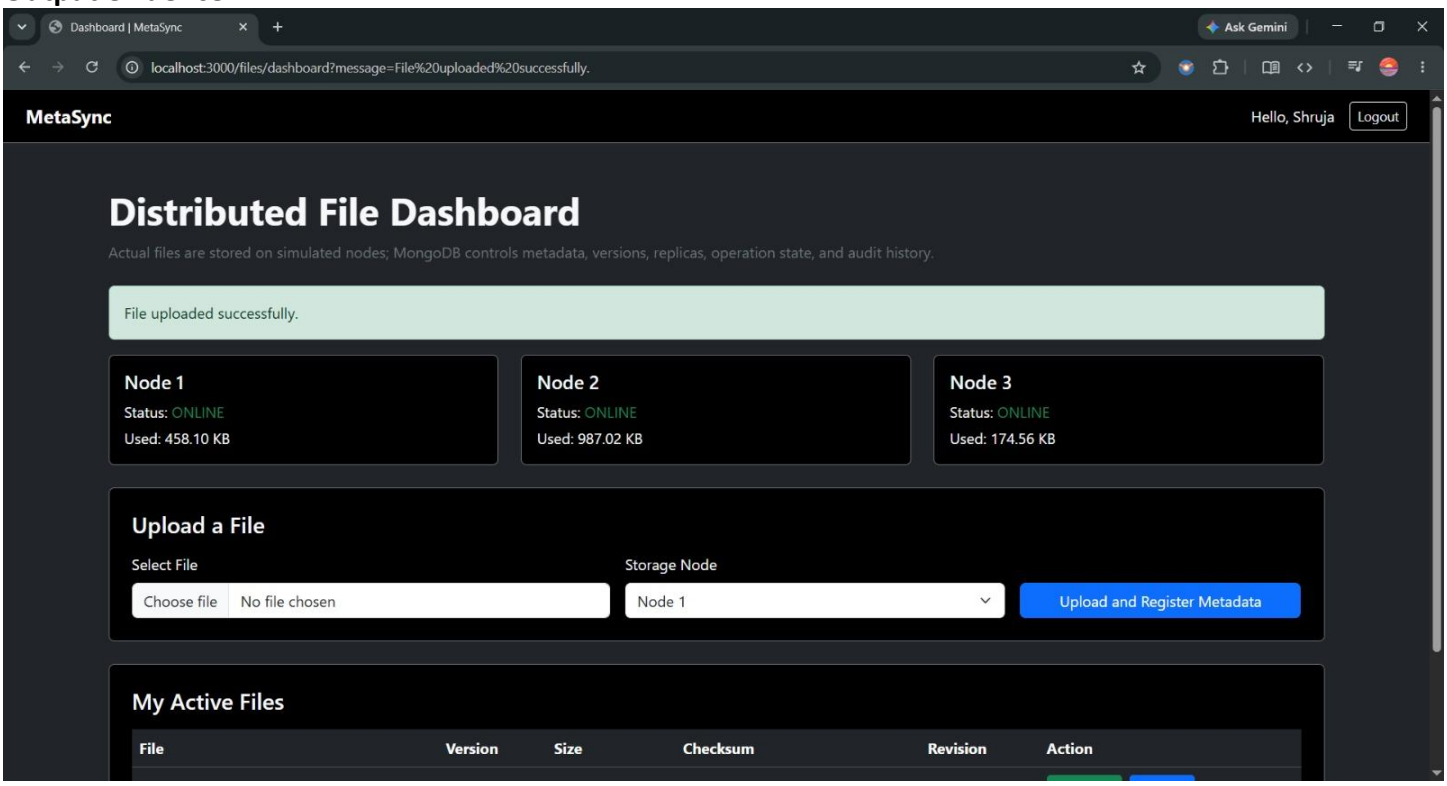
The upload is therefore recorded as both a storage operation and a database operation.

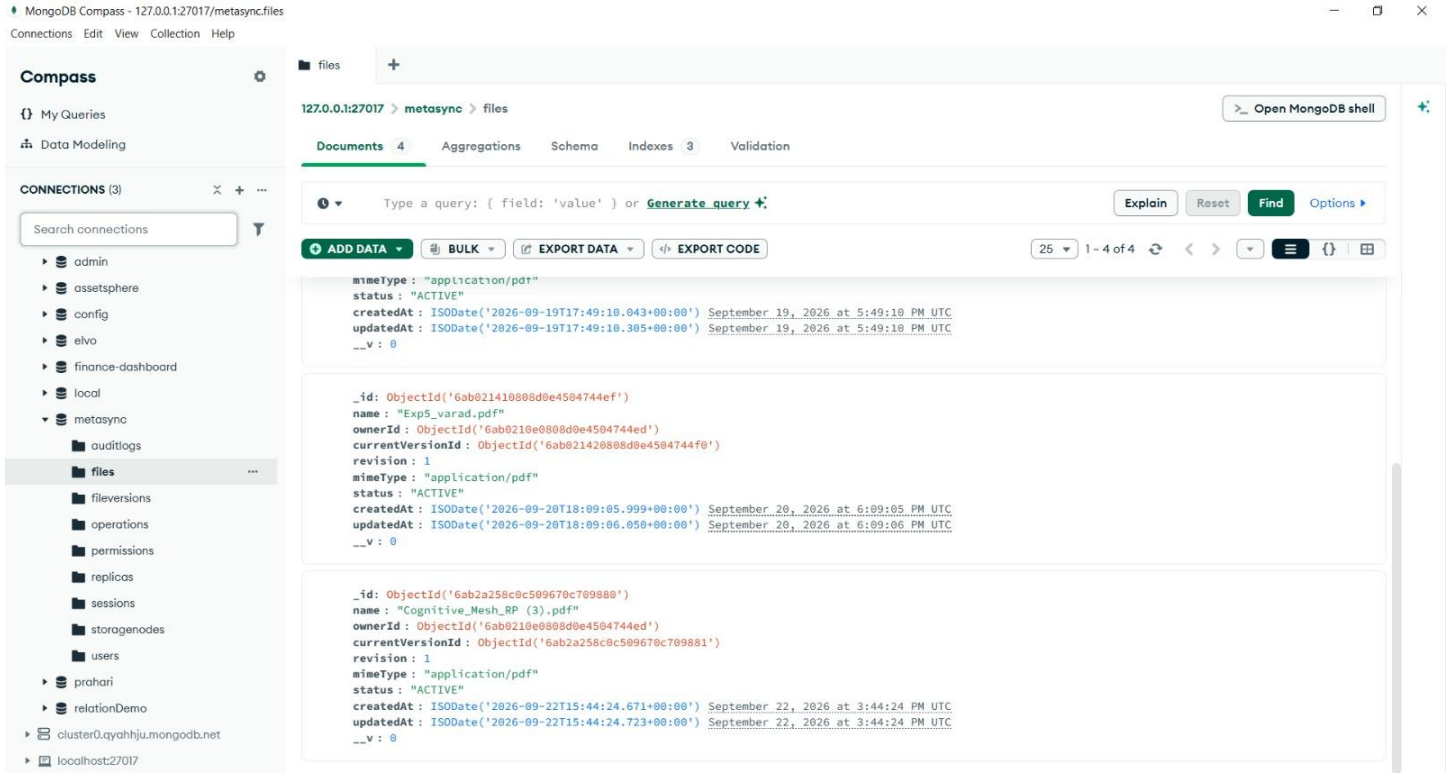
Main code snippet

```
const storedPath = path.join(node.path, uniqueName);
fs.renameSync(req.file.path, storedPath);

const file = await File.create({
  originalName: req.file.originalname,
  owner: req.session.userId,
  currentVersion: 1,
  storageNode: node._id
});
```

Output evidence




MongoDB Compass - 127.0.0.1:27017/metasync.files
Connections Edit View Collection Help
Compass
My Queries
Data Modeling
CONNECTIONS (3)
Search connections
admin
assetsphere
config
elvo
finance-dashboard
local
metasync
auditlogs
files
fileversions
operations
permissions
replicas
sessions
storagenodes
users
prahari
relationDemo
cluster0.qyahhju.mongodb.net
localhost:27017
files
127.0.0.1:27017 > metasync > files
Open MongoDB shell
Documents 4
Aggregations
Schema
Indexes 3
Validation
Type a query: { field: 'value' } or Generate query
Explain
Reset
Find
Options
ADD DATA
BULK
EXPORT DATA
EXPORT CODE
25
1 - 4 of 4
mimeType : "application/pdf"
status : "ACTIVE"
createdAt : ISODate('2026-09-19T17:49:10.043+00:00') September 19, 2026 at 5:49:10 PM UTC
updatedAt : ISODate('2026-09-19T17:49:10.305+00:00') September 19, 2026 at 5:49:10 PM UTC
__v : 0
_id: ObjectId('6ab021410808d0e4504744ef')
name : "Exp5_varad.pdf"
ownerId : ObjectId('6ab0210e0808d0e4504744ed')
currentVersionId : ObjectId('6ab021420808d0e4504744f0')
revision : 1
mimeType : "application/pdf"
status : "ACTIVE"
createdAt : ISODate('2026-09-20T18:09:05.999+00:00') September 20, 2026 at 6:09:05 PM UTC
updatedAt : ISODate('2026-09-20T18:09:06.050+00:00') September 20, 2026 at 6:09:06 PM UTC
__v : 0
_id: ObjectId('6ab2a258c0c509670c709880')
name : "Cognitive_Mesh_RP (3).pdf"
ownerId : ObjectId('6ab0210e0808d0e4504744ed')
currentVersionId : ObjectId('6ab2a258c0c509670c709881')
revision : 1
mimeType : "application/pdf"
status : "ACTIVE"
createdAt : ISODate('2026-09-22T15:44:24.671+00:00') September 22, 2026 at 3:44:24 PM UTC
updatedAt : ISODate('2026-09-22T15:44:24.723+00:00') September 22, 2026 at 3:44:24 PM UTC
__v : 0

6 Physical File Distribution Across Nodes

The actual file is saved locally in a node specific directory such as storage/node1, storage/node2, or storage/node3.

The selected storage node is saved as metadata, so MetaSync can later locate the correct physical file.

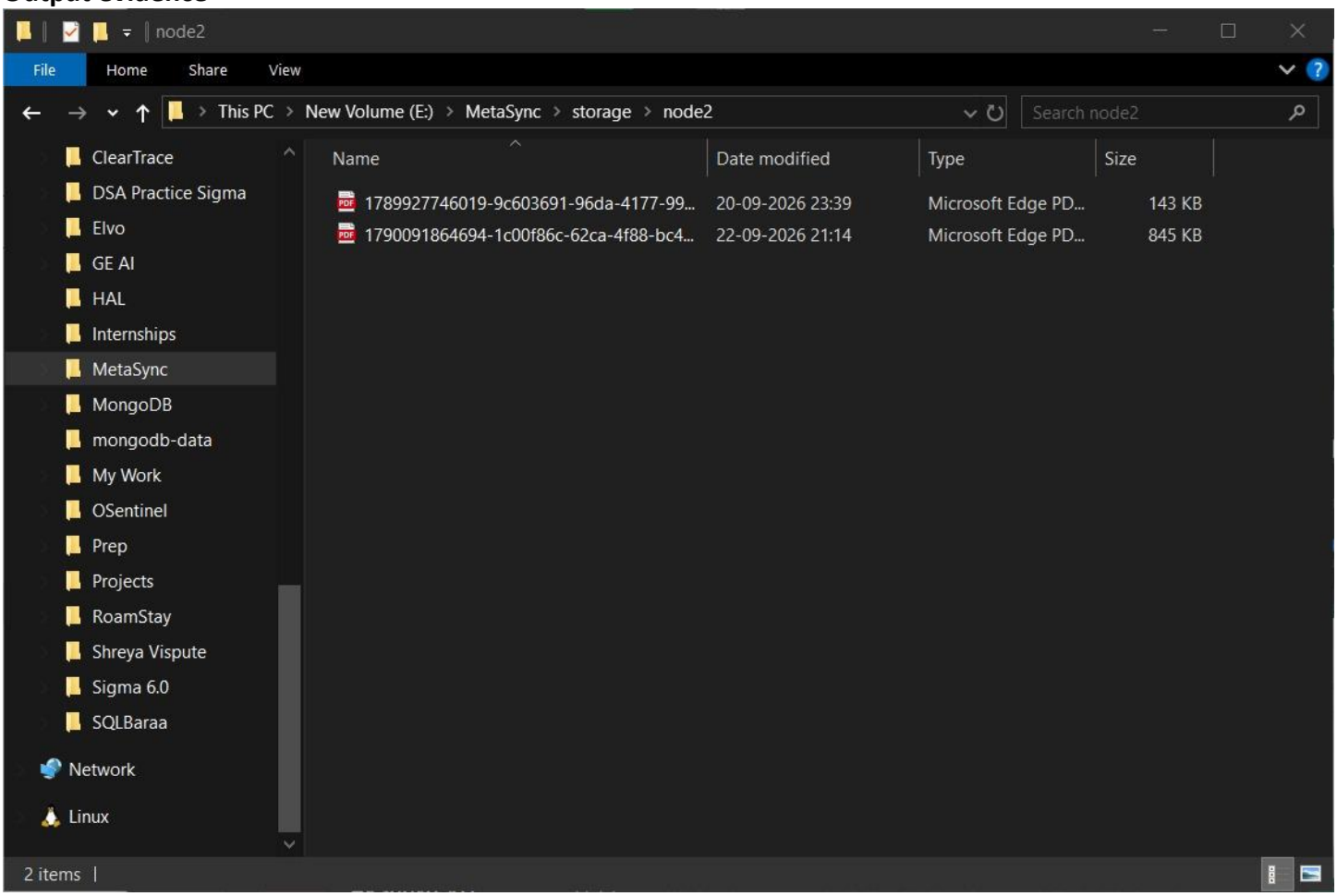
This separation is intentional: MongoDB manages the information about the file, while the node folder holds its bytes.

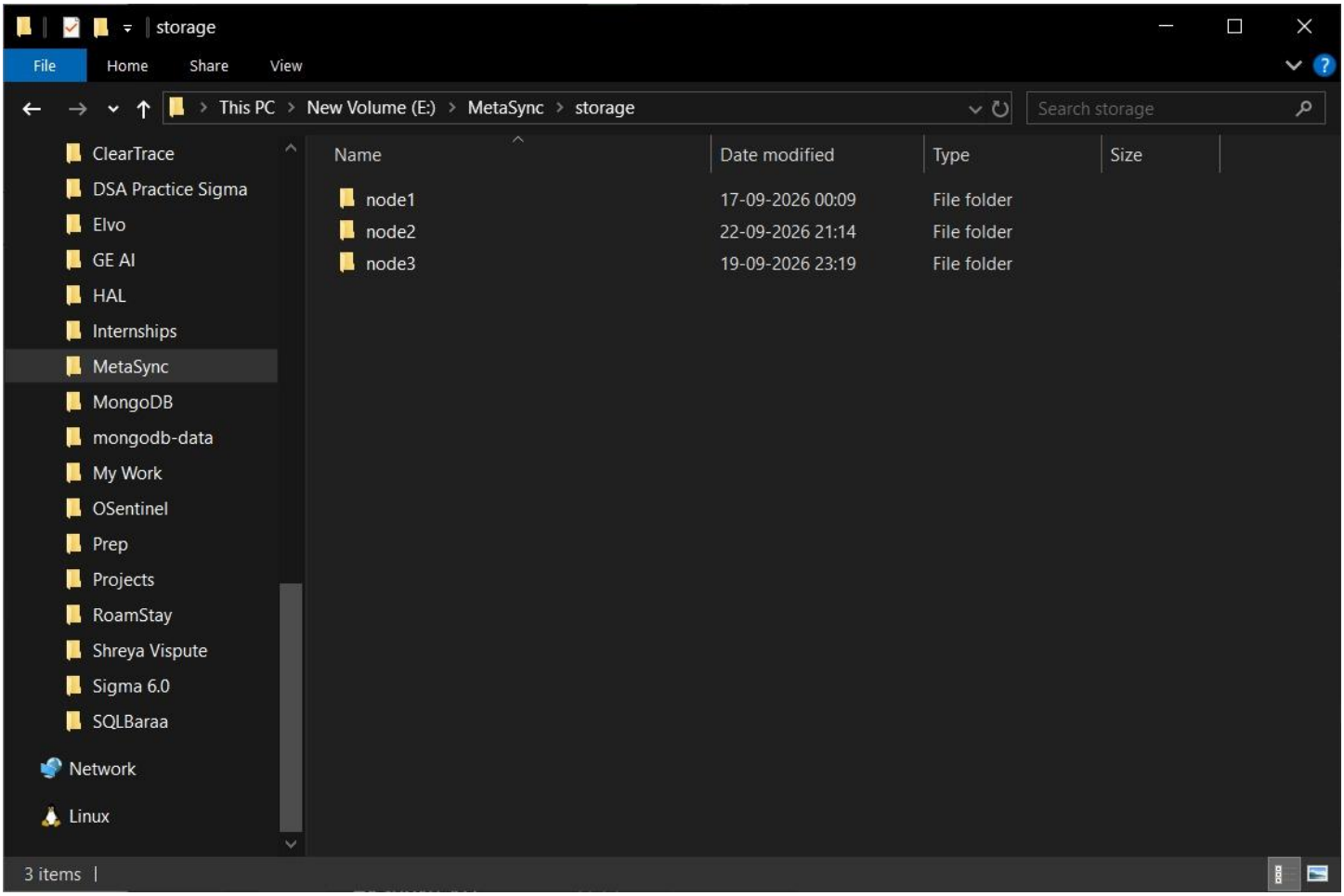
It demonstrates the central DBMS idea of metadata management for distributed storage.

Main code snippet

```
const nodeFolders = {
  1: "storage/node1",
  2: "storage/node2",
  3: "storage/node3"
};
const destination = path.join(nodeFolders[nodeNo], uniqueName);
```

Output evidence




storage
File
Home
Share
View
This PC
New Volume (E:)
MetaSync
storage
Search storage
ClearTrace
DSA Practice Sigma
Elvo
GE AI
HAL
Internships
MetaSync
MongoDB
mongodb-data
My Work
OSentinel
Prep
Projects
RoamStay
Shreya Vispute
Sigma 6.0
SQLBaraa
Network
Linux
Name
Date modified
Type
Size
node1
17-09-2026 00:09
File folder
node2
22-09-2026 21:14
File folder
node3
19-09-2026 23:19
File folder
3 items

7 MongoDB Metadata and Replica Record

MongoDB Compass displays the metadata created for every uploaded file.

The files collection stores the logical file record and the replicas collection records the node location.

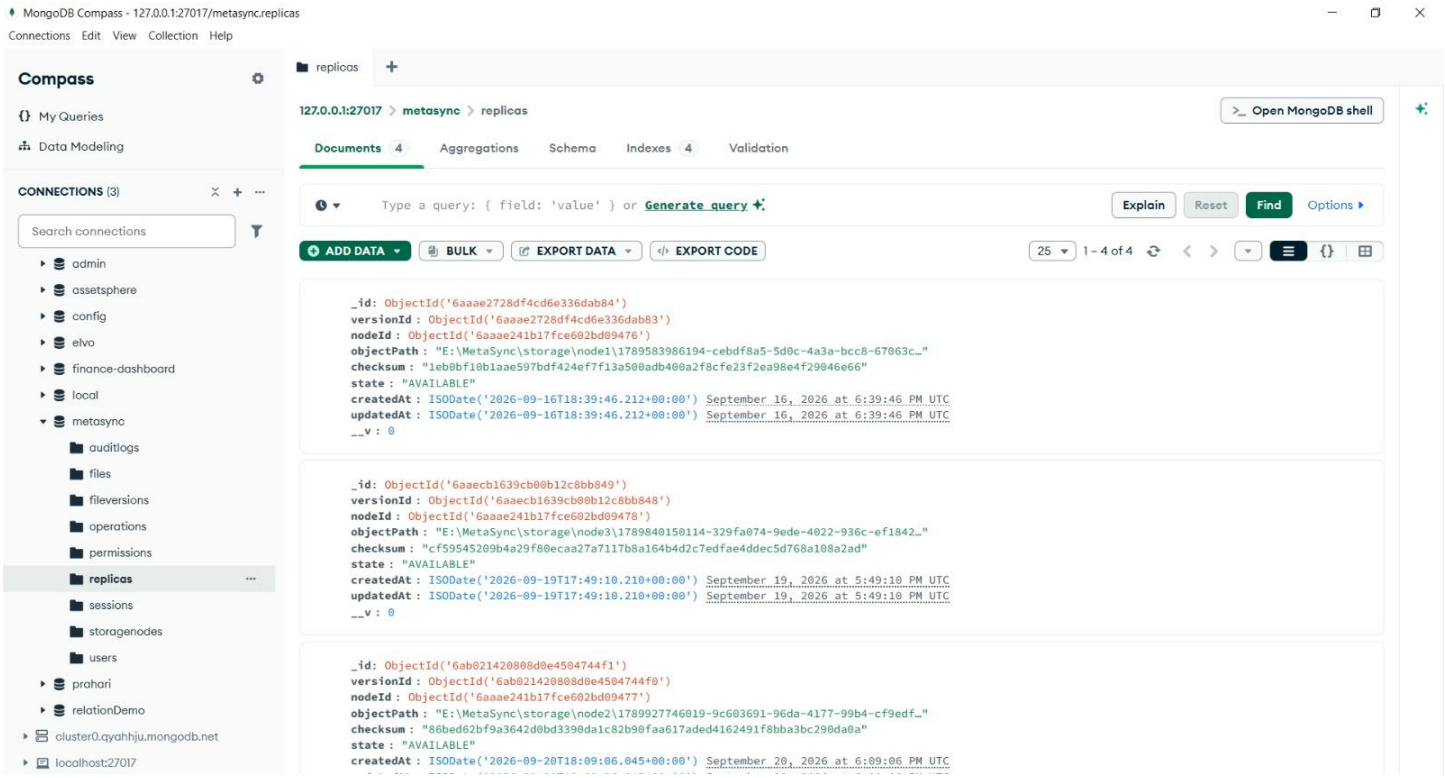
The fileversions collection maintains version history separately from the current file record.

This normalized by responsibility design makes metadata queries and recovery operations easier.

Main code snippet

```
await Replica.create({
  file:  file._id,
  node: node._id,
  filePath: storedPath,
  status: "ACTIVE"
});
await StorageNode.findByIdAndUpdate(node._id, {
  $inc: { usedSpace: req.file.size }
});
```

Output evidence



8 File Versioning and Restore

MetaSync keeps file versions as separate immutable records instead of overwriting old history.

When a new version is uploaded, a FileVersion document is created and the main File document points to the newest version.

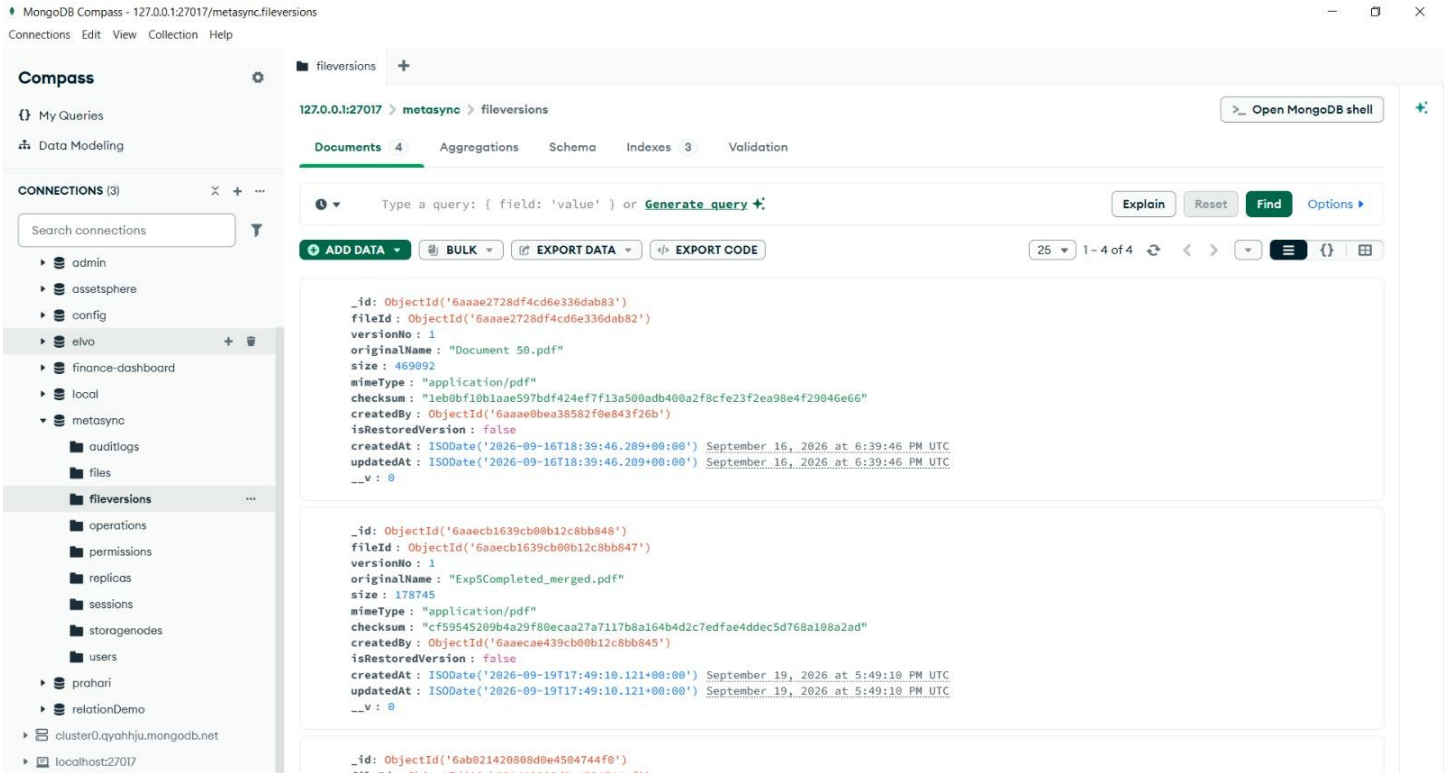
An earlier version can be restored, creating a new current version while retaining the old records.

This protects users from accidental replacement and provides a recoverable history.

Main code snippet

```
const version = await FileVersion.create({
  file: file._id,
  versionNumber: file.currentVersion + 1,
  filePath: storedPath,
  checksum
});
file.currentVersion = version.versionNumber;
await file.save();
```

Output evidence



9 Secure File Download

Users can download a registered file from the dashboard.

The controller checks the logged-in user and reads the physical path from the stored metadata.

The server then sends the stored file to the browser using a download response.

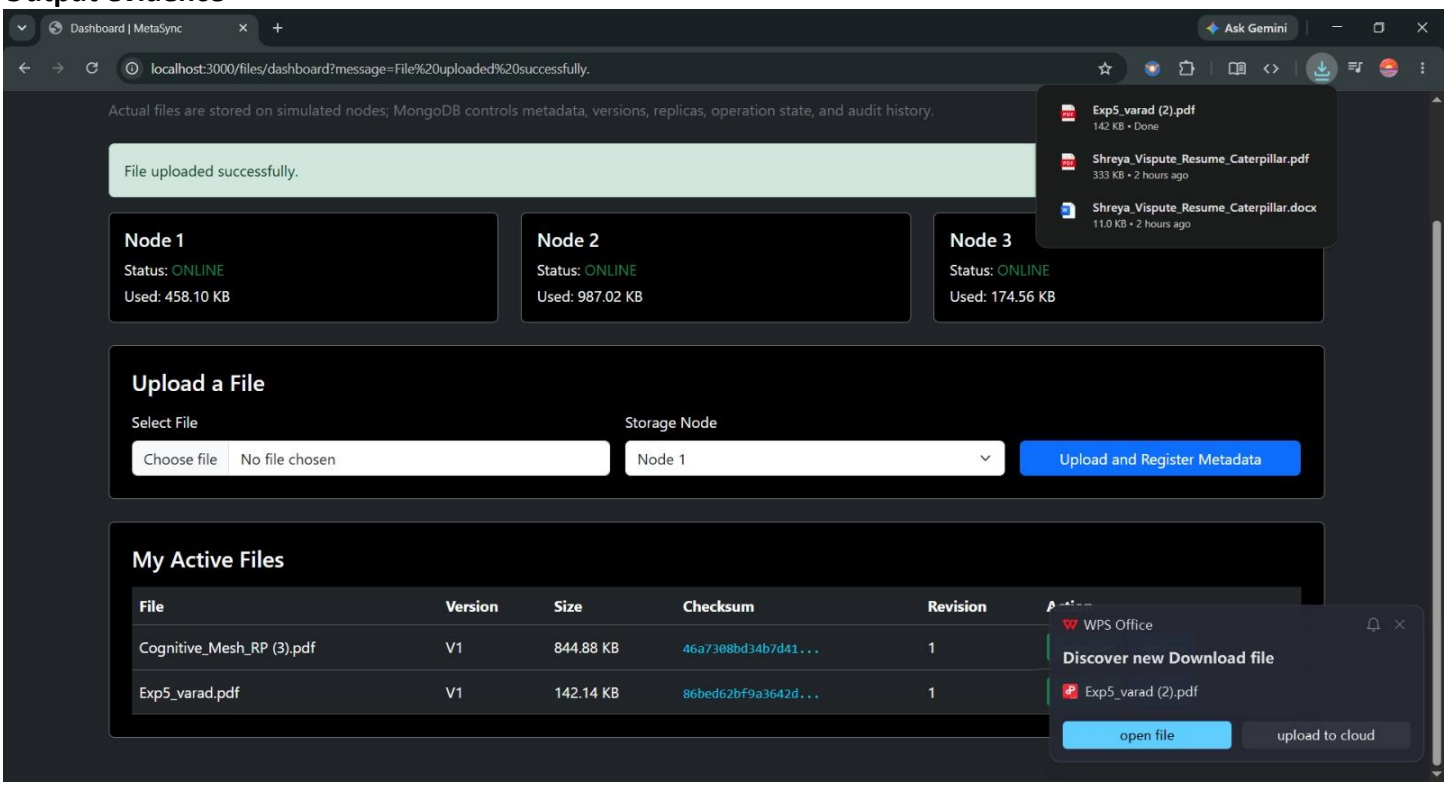
This demonstrates the link between MongoDB logical records and the file stored on a selected node.

Main code snippet

```
const file = await File.findById(req.params.id);
if (!file) return res.redirect("/files/dashboard");

res.download(file.currentPath, file.originalName);
await AuditLog.create({
  action: "DOWNLOAD", file: file._id, user: req.user._id
});
```

Output evidence



10 Operation Tracking and Audit Logs

Every important file activity is recorded for accountability and recovery analysis.

Operation documents record workflow state such as PENDING, COMPLETED, or FAILED.

AuditLog records who performed an upload, download, version restore, or permission related action and when it occurred.

This feature provides traceability, which is essential in a database controlled storage system.

Main code snippet

```
const operation = await Operation.create({
  type: "UPLOAD", status: "PENDING", user: req.user._id
});
// file and replica work executes here
operation.status = "COMPLETED";
await operation.save();
await AuditLog.create({ action: "UPLOAD", user: req.user._id, file: file._id });
```

Output evidence

